

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INFORMATICA



Tesi di laurea di I livello in
Informatica

Human-Computer Interaction & Music: Unity3D and Leap Motion Controller

Relatore:

Rocco Zaccagnino

Candidato:

Antonino Manuele Gargiulo

Mat. 05121 05566

ANNO ACCADEMICO 2019/2020

Dedicato agli amici di Lancusy

*Francesco Milano, Marco Villani, Francesco Gargiulo, Vincenzo Tortora,
Mario Paone, Gianluca Mannini, Daniele Perrella, Massimiliano Mauro.*

Scendi, buonasera.

INDICE

1	INTRODUZIONE	4
2	RELATED WORKS	8
3	BACKGROUND	13
3.1	Natural User Interfaces	13
3.2	Unity 3D	15
3.2.1	Assets	18
3.2.2	GameObjects & Components	19
3.2.3	Scene	20
3.2.4	Scripting: La classe MonoBehaviour	20
3.2.5	Audio in Unity	23
3.3	Gesture Recognition	24
3.4	Leap Motion	26
3.5	Fondamenti di Acustica	27
3.6	Digital Audio	28
3.7	Artificial Neural Networks	31
3.8	Paradigmi di apprendimento	32
4	MarcoSmiles 2.0	35

INDICE

5	MODULO UNITY	37
5.1	Data Representation	37
5.2	Dataset	38
5.3	Gestione Progetto Unity	39
5.4	Gestione Salvataggio dati	41
5.4.1	Gestione Directory	41
5.4.2	Registrazione e Salvataggio Configurazioni Mani	42
5.5	Interfacciamento Python e Unity	47
5.6	GUI	50
6	CONCLUSIONI	55
6.1	RISULTATI E DISCUSSIONI	55
6.2	Lavori futuri	56

CAPITOLO 1

INTRODUZIONE

Viviamo in un'epoca in cui la tecnologia ha completamente rivoluzionato molti degli aspetti della vita giornaliera di un individuo. In un contesto simile, fornire accesso agli strumenti informatici di uso comune ad ogni tipo di utente, è di fondamentale importanza. Requisiti come l'usabilità, intesa come la facilità di interazione tra l'uomo e uno strumento, e accessibilità, ovvero la caratteristica di un servizio di essere fruibile da tutte le categorie di utenti, rappresentano dei punti cardine in fase di progettazione di un sistema informatico, e sono oggetto di studio della *Human Computer Interaction* (HCI). Il costante sviluppo in questi ambiti ha portato alla nascita di strumenti che possano permettere l'utilizzo dei dispositivi elettronici a chiunque, scavalcando barriere derivanti da problemi quali il gap inter-generazionale, difficoltà d'utilizzo, mancanza di accessibilità e scarsa attrattività. In particolare negli ultimi anni sono stati esplorati nuovi modi di interagire con i computer, progettando nuove interfacce basate sull'utilizzo di ambienti 3D, gestures e contextual awareness, al fine di rendere l'interazione più semplice, intuitiva e accattivante. In tale contesto, sono state oggetto di numerosi studi negli scorsi anni le *Natural User Interfaces* (NUI), descritte alla presentazione della conferenza "Predicting the Past" del 2008 come "la

prossima fase evolutiva dopo il passaggio dall'interfaccia a riga di comando (CLI) all'interfaccia grafica utente (GUI)” [1]. L'obiettivo principale delle NUI, è quello di superare le classiche interazioni fisiche legate all'utilizzo di mouse e tastiera, per sostituirle con un più ampio insieme di modalità di comunicazione innovative, che prevedano l'utilizzo di azioni, gesti, e movimenti naturali per l'utente, al fine di controllare la macchina che sta utilizzando [2]. La popolarità di queste interfacce innovative è in continua crescita, soprattutto grazie alla diffusione di dispositivi low-cost per il tracking dei movimenti, come il *Leap Motion* o il *Kinect*, e dispositivi che supportano il riconoscimento della voce come *Amazon Alexa*. Nonostante la già grande popolarità nel mercato dei videogiochi, queste tecnologie sono state impiegate in vari settori quali la riabilitazione, la robotica, e l'home automation.

L'obiettivo di questa tesi è quello di utilizzare questo tipo di interfacce, insieme ad un dispositivo per il riconoscimento dei movimenti delle mani, nell'ambito della *Computer Music*. La computer music o musica informatica è un ramo della musica elettroacustica che impiega sistematicamente la programmazione su elaboratore numerico. In senso più esteso, qualunque produzione musicale che utilizzi il mezzo informatico può definirsi computer music [3]. Pietro Grossi, pioniere italiano dell'ingegneria, nel 1967, alle prime esperienze ufficiali di computer music presso la Olivetti General Electric a Pregnana Milanese dichiarò: “...Ho preso il quinto Capriccio di Paganini, ho perforato le schede e, fatto il pacco di schede, consegnate al computer, il computer le suonava. Il computer ha suonato subito alla perfezione il testo che gli avevo dato, e immediatamente dopo, ho potuto fare di quei suoni quello che volevo. Questo per me era un salto incredibile. È stata un'emozione straordinaria...” [4]. Quest'ambito è in continua evoluzione, e negli ultimi anni diversi musicisti hanno mostrato interesse nella possibilità di utilizzare controller che siano in grado di personalizzare il modo in cui ci si esprime “musicalmente”. Un esempio degno di nota è la performance eseguita dalla nota cantante americana Ariana Grande nel 2015 [5], in cui fa uso dei guanti Mi.Mu Gloves [6] per armonizzare la melodia che sta cantando,

1. INTRODUZIONE

mettendo in luce le grandi potenzialità derivanti dall'utilizzo di un controller per l'espressività.

Per la maggior parte degli strumenti musicali reali, è possibile rappresentare le note musicali in termini di configurazioni della mani. Ogni configurazione contiene informazioni circa la posizione e la direzione di mano e dita, ad un certo istante di tempo. Una modalità semplice e naturale di associare gesti dell'utente alla riproduzione di musica real-time è proprio attraverso l'associazione di configurazioni delle mani a delle specifiche note musicali. Su questo principio è basato il sistema MarcoSmiles [7], uno strumento rivolto principalmente a persone affette da disabilità motorie, ma utilizzabile anche da chi è interessato all'ambito musicale o semplicemente per attività di intrattenimento, che consente di poter suonare uno strumento virtuale attraverso dei semplici gesti delle mani, senza l'uso di uno strumento musicale reale, e in maniera del tutto personalizzabile. Per il funzionamento del sistema è necessario un sensore LeapMotion che avrà il ruolo di acquisire informazioni circa la posizione della mani dell'utente, per poi fornirle ad una Rete Neurale Artificiale (ANN) opportunamente addestrata che provvederà a realizzare l'associazione posizione - nota musicale real-time.

Questo lavoro di tesi ha come obiettivo quello di superare i limiti di tale progetto proprio sulla parte dell'interfaccia, definendo ed implementando un'interfaccia grafica basata su Unity3D che permetta un accesso più performante e naturale. La scelta di Unity3D apre poi una prospettiva futura verso la realtà aumentata, cioè verso la realizzazione di una piattaforma in cui l'utente è completamente "immerso" nella performance musicale, con l'illusione di poter interagire fisicamente con lo strumento virtuale definito a partire dalle proprie scelte gestuali. Nel Capitolo 2 discuteremo di alcuni lavori rilevanti in questo settore, tracciando un percorso storico sull'evoluzione degli strumenti musicali non convenzionali. Nel Capitolo 3 verranno descritti i principali ambiti che verranno trattati in questo lavoro. Nel quarto capitolo verrà fornita una descrizione del sistema e degli obiettivi che si pone. Il quinto capitolo è dedicato allo sviluppo del modulo oggetto di questa tesi. Infine, nel sesto

1. INTRODUZIONE

capitolo verranno discussi i risultati ottenuti, e le possibili prospettive sui lavori futuri per estendere il sistema.

CAPITOLO 2

RELATED WORKS

Negli ultimi anni sono stati effettuati molti studi che hanno evidenziato una vasta scelta di possibilità nel produrre musica in maniera non convenzionale, attraverso strumenti elettronici sia fisici che virtuali. Tuttavia per comprendere al meglio la storia e l'evoluzione degli strumenti musicali accessibili è necessario fare un passo indietro; quando l'utilizzo dell'elettricità era ancora nelle prime fasi di sperimentazione, e la lampadina doveva ancora essere inventata, alcune persone erano già a lavoro per utilizzare l'elettricità nel processo di creazione del suono. A causa della mancanza delle tecnologie di registrazione, questi primi esperimenti furono progettati esclusivamente per performance live [8]. Nel 1919 è nato il *Theremin* [9], il primo strumento che non richiede contatto fisico con l'esecutore. Il Theremin è stato inventato dal fisico sovietico Lev Sergeevič Termen nel 1919, e si basa sull'utilizzo di oscillatori elettronici che lavorano in isofrequenza al di fuori dello spettro udibile. Il musicista provoca un'alterazione degli oscillatori inserendo le sue mani nel campo d'onda, producendo dei suoni sul principio fisico del battimento, questa volta nel campo delle frequenze udibili. Lo strumento è sostanzialmente composto da 2 antenne annesse ad un contenitore in cui alloggia l'elettronica dello strumento. Le antenne controllano rispettivamente

2. RELATED WORKS

frequenza e intensità del suono prodotto. Il Theremin ha rappresentato una vera e propria rivoluzione nell'ambito della musica, ed è stato utilizzato negli anni successivi in diversi contesti e in numerosi generi musicali.

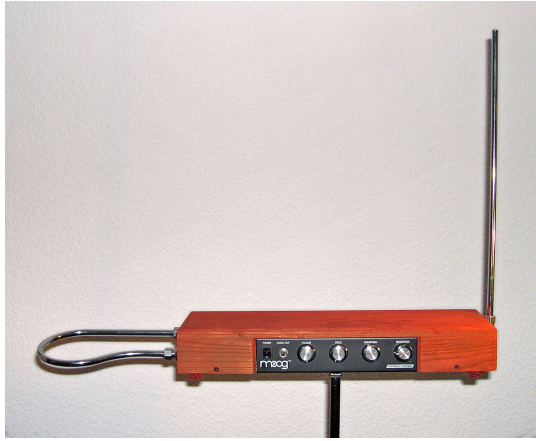


Figura 2.1: Theremin [9]

Verso la prima metà degli anni 70 è stato sviluppato l'iconico *Mellotron* [10], uno strumento a tastiera pensato per riprodurre strumenti musicali tipici delle orchestre sinfoniche, e voci umane. Questo strumento può essere considerato come il padre dei campionatori elettronici, comparsi all'inizio degli anni 80. Il mellotron genera i suoni tramite nastri registrati. Quando viene premuto un tasto, il nastro collegato viene spinto sulla testina di riproduzione, come in un registratore a nastro. Finché il tasto resta abbassato, il nastro scorre sulla testina e il suono viene riprodotto. Quando il tasto viene rilasciato, una molla fa tornare indietro il nastro alla sua posizione di riposo. In seguito ai progressi dell'industria elettronica, furono prodotti numerosi strumenti elettronici di vario genere, tra i quali possiamo ricordare il *Suzuki Omnichord* [11]. Il suo funzionamento è semplice: la mano sinistra preme i tasti che formano gli accordi, mentre la destra sfiora il touchpad che sostituisce le corde. L'omnichord offre altre possibilità per ottenere suoni, anche utilizzando soltanto la tastiera. È uno dei primi strumenti elettronici ad essere considerato accessibile, e viene utilizzato tutt'ora in musicoterapia. Attualmente nell'ambito della Computer Music rivestono un ruolo di grande importanza sensori di varie tipologie. Tra i più utilizzati troviamo i sensori

2. RELATED WORKS

tattili, fotocamere e accelerometri. Un importante esempio è rappresentato dal guanto *KAiKU* [13], un sensore capacitivo indossabile, che risponde alla prossimità e al contatto di conduttori, come il corpo umano.



Figura 2.2: Kaiku Glove durante una performance musicale [13]

Il sensore viene principalmente utilizzato come controller MIDI, quindi può essere utilizzato in vari contesti, e dispone di un software dedicato che permette di emulare diversi tipi di strumenti. Un tipico esempio di utilizzo consiste nel mappare delle note a delle zone del guanto, che possono essere sfiorate con le dita o altri oggetti conduttori per inviare un segnale MIDI al dispositivo a cui è connesso il guanto, in modo che venga suonata la nota desiderata. Sono presenti anche altri tipi di sensori indossabili, e vengono spesso utilizzati per la riabilitazione e la musicoterapia. Un altro strumento usato in questi due ambiti è il software *AUMI* [12]. Quest'ultimo permette all'utente di poter riprodurre suoni e frasi musicali attraverso movimenti e gestures, sfruttando la fotocamera del dispositivo sul quale è in uso. Il software prevede un approccio personalizzabile attraverso l'utilizzo di effetti e la possibilità di variare i suoni utilizzati. Il sistema è progettato principalmente per essere utilizzato da bambini che hanno disabilità fisiche. Molto diffuso è anche il *Kinect* [14], un sensore sviluppato da Microsoft per le console Xbox. Il Kinect si avvale di una telecamera RGB e di una telecamera a radiazione infrarossa con un risoluzione di 480p ciascuna, ed una capacità di lettura del movimento a 30fps (frames per second). Per il calcolo della lontananza degli oggetti, invece, Kinect utilizza di uno scanner 3D, in combinazione con gli infrarossi, che però è dotato di una risoluzione di 320×240 pixel. Kinect dispone inoltre di quattro microfoni utilizzati dal

2. RELATED WORKS

sistema per l'elaborazione dati dell'ambiente in cui ci si trova, che permettono la cancellazione dell'eco e riduzione del rumore. Il Kinect è stato utilizzato in numerosi progetti di vario genere nell'ambito della Computer Music, spesso facendo uso dell'Hand Gesture Recognition. Un esempio consiste nel lavoro effettuato dalla National Taiwan University of Science and Technology [15], ovvero un sistema capace di simulare la conduzione musicale in maniera real-time, sfruttando un classificatore. Il sensore utilizzato nel nostro sistema è il *LeapMotion Controller*. L'applicazione del sensore LeapMotion nell'ambito della Computer-Music è stata oggetto di diversi studi. In primo luogo è stata esplorata la possibilità di utilizzare il LeapMotion assieme al protocollo *MIDI*, sfruttando la Gesture-Recognition al fine di inviare segnare segnali per modulare determinati parametri all'interno di una DAW o di VST. Un esempio di questo tipo di applicazione è lo strumento AeroMIDI [16], che permette di creare un'interfaccia completamente personalizzabile, in cui è possibile mappare parametri di un Virtual Instrument o di qualsiasi altra funzione, a gesti delle mani, con la possibilità di controllarne fino a 6 per mano. Il sistema AeroMIDI fa parte della categoria dei numerosi strumenti che utilizzano il LeapMotion come controller MIDI, e garantiscono all'utente una maggiore espressività durante la performance musicale. Uno studio sull'Gesture Recognition utilizzando il LeapMotion è stato effettuato dalla Dalian University of Technology [17], e ha evidenziato la possibilità di sviluppare un classificatore in grado di riconoscere accuratamente gesture scelte dall'utente in fase di training. Un altro progetto che utilizza il LeapMotion nell'ambito della Computer - Music è *Virtual Piano* [18]. Si tratta di un software basato sulla gesture recognition, che permette di poter suonare un pianoforte virtuale attraverso delle gesture delle mani predefinite, e dispone di alcune semplici canzoni da poter suonare.

2. RELATED WORKS

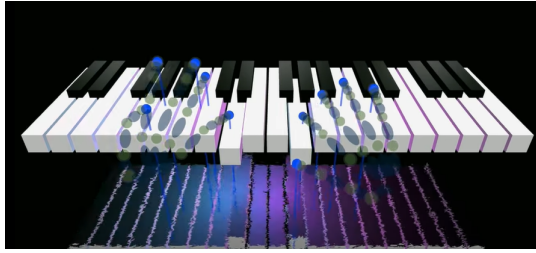


Figura 2.3: Virtual Piano [18]

E' importante sottolineare la principale differenza fra i sistemi sopracitati rispetto a *MarcoSmiles*; la configurazione delle mani associata ad una determinata nota è scelta dall'utente in base alle sue preferenze. L'utente dovrà allenare il sistema con i gesti che intende utilizzare, per poi successivamente utilizzarli per suonare in modo naturale, come se si trattasse di uno strumento reale.

CAPITOLO 3

BACKGROUND

3.1 Natural User Interfaces

Come già accennato nel paragrafo introduttivo, le Natural User Interfaces sono interfacce invisibili all'utente, caratterizzate da interazioni semplici e di facile apprendimento. La parola “Natural” si riferisce proprio all'obiettivo delle NUI, cioè rendere l'interazione con l'utente del tutto naturale, attraverso l'utilizzo di capacità umane quali tatto, vista, utilizzo della voce, movimento, fino ad arrivare a funzioni cognitive più complesse come percezione ed espressione. Un altro aspetto importante riguardo le NUI consiste nella selezione di gestures che l'utente conosce e utilizza durante la vita di tutti i giorni. Ciò avviene in modo da ridurre il carico cognitivo dell'utilizzatore, cioè lo sforzo richiesto nel pensare o effettuare un ragionamento prima di usare una determinata gesture all'interno dell'interfaccia, in maniera tale da eliminare qualsiasi tipo di distrazione. Da tenere in grande considerazione è anche il contesto in cui la NUI viene utilizzata. Durante la fase di progettazione bisogna tener conto dell'impossibilità nel creare un' interfaccia che risulti naturale in tutti i contesti e per tutti gli utenti. Di conseguenza la scelta delle gestures deve essere effettuata alla luce del contesto in cui l'interfaccia viene utilizzata,

e tenendo conto dello skill level dell'utilizzatore medio. Le NUI presentano diverse caratteristiche [19]:

- **User-centered:**

Devono adattarsi alle necessità dell'utente. In un tradizionale approccio uomo-macchina, le persone vengono considerate come operatori che si adattano alla macchina; in un approccio moderno le persone vengono considerate come utenti che dialogano con la macchina, tuttavia senza averne un controllo effettivo; in un approccio che utilizza le NUI le persone sono partecipanti attivi, e la macchina risponde alle varie interazioni dell'uomo.

- **Multi-channel:**

Le interfacce Multi-channel fanno uso di uno più sensori per determinare le intenzioni dell'utente durante l'interazione, al fine di migliorarla e renderla più naturale. L'utente può comunicare attraverso diverse modalità quali vista, tatto, udito, olfatto, ed equilibrio. I "mezzi" di cui si serve sono mani, bocca, occhi, piedi etc. Nelle classiche interazioni uomo macchina, sono coinvolti esclusivamente vista e tatto, attraverso gli occhi alle mani. Per rendere l'interazione più efficiente e al fine di non affaticare i "mezzi" dell'utente, quest'ultimo dovrebbe poter scegliere il miglior mezzo per comunicare con la macchina.

- **Imprecisione:**

Una tecnologia che offre un'interazione precisa riesce a comprendere l'intenzione dell'utente nella sua interezza. Spesso le azioni o i pensieri delle persone non risultano molto precisi, dunque la macchina deve essere in grado di capire le richieste dell'utente, tenendo conto anche del possibile margine d'errore.

- **Voice-based interaction:**

Il linguaggio umano rappresenta il modo più naturale ed efficiente per condividere informazioni. In media durante la giornata, circa 75% della

comunicazione effettuata un individuo avviene attraverso la voce. Inoltre il riconoscimento di segnali audio è significativamente più veloce del riconoscimento di segnali visivi.

Si può dunque dire che la voce rappresenta il più importante mezzo di interazione tra una macchina e un essere umano.

- **Behavior-based Interaction:**

Nel processo di interazione, oltre alla voce, le persone utilizzano anche i movimenti del corpo per esprimersi. L'interazione uomo-macchina basata sul comportamento consiste nella capacità di una macchina di riconoscere il comportamento umano attraverso posizionamento, movimento ed caratteristiche espressive del corpo umano, e nel rispondere in maniera "intelligente" a variazioni di esse. Con questo tipo di interazione il comportamento dell'utente può essere predetto dalla macchina.

Il noto fondatore di Microsoft, Bill Gates, ha dichiarato in merito alle NUI: "Fino ad ora ci siamo sempre dovuti adattare ai limiti imposti della tecnologia e adattare il modo in cui lavoriamo con i computer ad un insieme di procedure e convenzioni. Attraverso le NUI, per la prima volta saranno i computer ad adattarsi ai nostri bisogni e preferenze, e per la prima volta gli essere umani interagiranno con le macchine in maniera del tutto naturale" [2].

3.2 Unity 3D

Il nostro sistema è basato su Unity3D [21], un motore grafico multi-piattaforma che consente lo sviluppo di videogiochi e altri contenuti interattivi, quali visualizzazioni architettoniche o animazioni 3D in tempo reale. Unity permette all'utente di creare ambienti sia in 2D che in 3D. Uno screening del 2018 rileva che Unity è stato utilizzato per la creazione di circa la metà dei giochi mobile disponibili sul mercato e del 60% dei giochi in realtà aumentata e VR. È inoltre stato adottato da altri ambiti industriali che non

3. BACKGROUND

rientrano in quello del settore dei videogiochi, come cinemagrafia, architettura, ingegneria e costruzione. Un vantaggio non di poco conto è che Unity mette a disposizione versioni gratuite per studenti e per privati con entrate inferiori ai 100.000\$ annuali.

L'Engine Unity3D, rende agevole la disposizione di livelli, la creazione di menu, l'esecuzione di animazioni e l'organizzazione dei progetti. Inoltre, integra al proprio interno la possibilità di scrivere ed eseguire scripts, scritti in C#.

L'interfaccia grafica è molto ben organizzata e i pannelli di lavoro sono totalmente personalizzabili. E' proprio attraverso la selezione ed il trascinamento dei pannelli che si attiva tale personalizzazione.

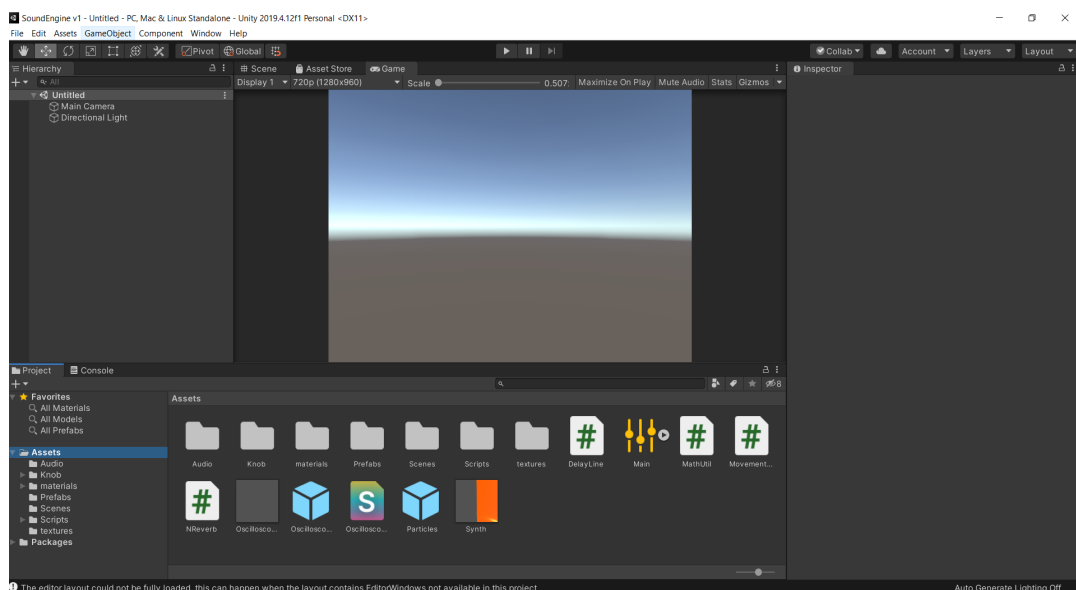


Figura 3.1: Scena vuota Unity3D

Questi i principali pannelli all'interno di Unity:

- Il pannello “Project” é quello in cui tutti gli assets del progetto vengono immagazzinati. Da qui è possibile organizzare i file che usiamo all'interno di un progetto.

Ogni volta che importiamo degli assets appariranno qui.

3. BACKGROUND

- Il pannello “Hierarchy” è quello in cui gli assets vengono organizzati all’interno di una scena. Ogni Asset, può essere trasportato dal pannello “Project” a quello “Hierarchy”, in modo tale da inserire assets all’interno di una scena. Gli elementi all’interno di questo pannello sono chiamati GameObjects.
- Il pannello “Inspector” permette di ispezionare e regolare tutti gli attributi di un GameObject. Da questo pannello si possono regolare per un GameObject, ad esempio, la posizione, la rotazione, le proprietà di gravità all’interno del gioco, e tutte le altre caratteristiche. Questo è fatto attraverso l’ausilio di Components, ossia, componenti che ci permettono di specificare determinate proprietà per il GameObject selezionato. Anche gli scripts vengono trattati come se fossero Components e, dunque, aggiunti tramite questo pannello.
- Il pannello “Scene” permette di osservare e di arrangiare gli assets all’interno del gioco e dello spazio 3D. Da qui, è possibile spostarci all’interno dello spazio 3D utilizzando il mouse.
- Il pannello “Console” è letteralmente la console di Unity. Molto utile in fase di debugging degli scripts.

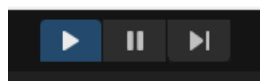


Figura 3.2: Pulsanti Play

Per far partire il proprio gioco si preme sul pulsante Play posto nella parte alta della schermata. Si può mettere in pausa il gioco (usando il pulsante Pause) in modo tale da ispezionare la scena. Inoltre è possibile passare allo step successivo del gioco, utilizzando il pulsante Step.

3. BACKGROUND

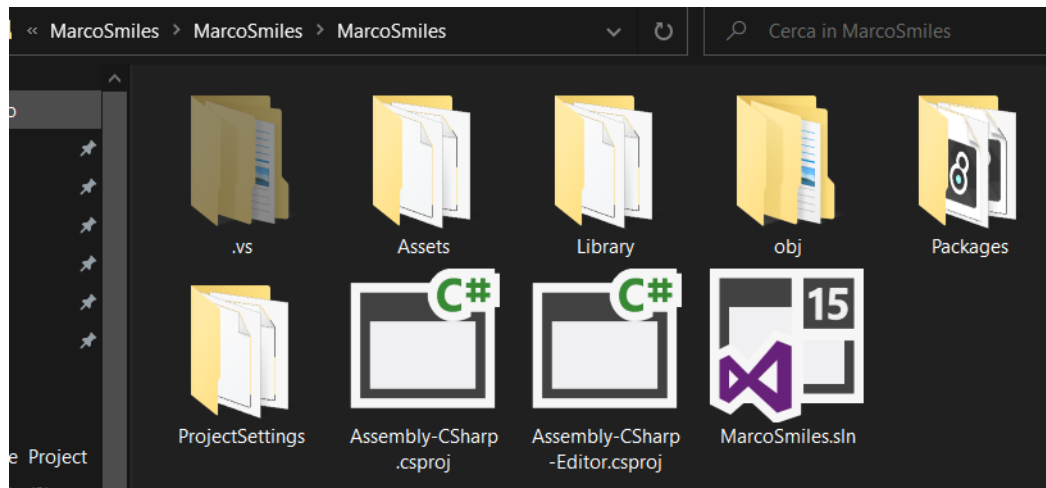


Figura 3.3: La directory di un progetto Unity

In Unity, un progetto è una cartella ordinaria, contenente ogni risorsa utilizzata dal gioco. Infatti, come risultato della creazione di un progetto nuovo, avremo una cartella che contiene delle sottocartelle chiamate rispettivamente “Assets”, “Library” e “ProjectSettings”.

3.2.1 Assets

Gli Assets sono una qualsiasi risorsa utilizzata dal gioco. Questo include modelli 3D, Materials, Textures, Audio, Scripts, Fonts etc..

Unity gestisce in modo robusto ed intelligente l’importazione di Assets in un progetto. Gestisce al meglio la maggior parte dei formati di file, risolvendo uno dei difetti che tradizionalmente ha caratterizzato vecchi game engine, ossia quello di essere vincolati all’estensione del file quando si cerca di importarlo all’interno di un progetto.

Unity ha funzionalità built-in che permettono di creare semplici oggetti come Sfere o Cubi, ma quando c’è la necessità di lavorare con oggetti più complessi, l’ideale è importare modelli 3D all’interno del proprio progetto. L’Engine riesce a gestire la maggior parte dei formati 3D, inclusi Maya, 3D Studio Max, Blender e FilmBox, mantenendo tutti i materiali e le textures intatte. Stesso discorso è valido per ciò che riguarda formati di immagini, supportando PNG, JPEG, TIFF e addirittura file PSD con livelli Photoshop, e per i formati di

file audio, supportando formati quali WAV, AIFF, MP3 e OGG.

Una lista completa di tutti i formati che possono essere importati all'interno di Unity la si può trovare al seguente link:

<http://unity3d.com/unity/editor/importing>

E' opportuno notare che Unity mette a disposizione un Assets Store, dove si possono acquistare o scaricare gratuitamente Assets e risorse per i propri progetti.

3.2.2 GameObjects & Components

Il concetto di **GameObject** è probabilmente il concetto più importante all'interno dell'editor di Unity. Ogni oggetto all'interno del gioco è un **GameObject**. Nonostante ciò, un **GameObject** non può fare nulla all'interno del gioco se non gli si conferiscono proprietà. Possiamo descrivere un **GameObject** come una scatola che permette di contenere funzionalità. Queste funzionalità sono descritte dai **Components**. A seconda di che tipo di oggetto si vuole creare, si aggiungono diverse combinazioni di **Components** all'interno di un **GameObject**.

Un **GameObject** contiene sempre una Component di tipo Transform che non è possibile rimuovere. Questa rappresenta la posizione e l'orientamento dell'oggetto.

Altre Components che danno funzionalità agli oggetti possono essere aggiunte dal menu rinominato "Component" o da uno script.

E' importante notare che esistono degli oggetti pre-costruiti chiamati Primitive Objects. Questi si trovano nel menu GameObject/3D Object e permettono di creare degli oggetti semplici come una camera, una sfera o un cilindro.

Per default, ogni scena viene inizializzata con un GameObject chiamato Main Camera. Questo oggetto è configurato per comportarsi come camera principale all'interno del gioco. Contiene una componente di tipo Transform, una di tipo Camera e un'altra di tipo AudioListener che permette di percepire i suoni che si trovano nell'ambiente 3D di una scena.

3.2.3 Scene

Le scene rappresentano determinati ambienti del gioco. Sono i “luoghi” che contengono gli Assets e descrivono la schermata del gioco. Mentre il pannello “Scene” è ideale per arrangiare gli Assets all’interno dello spazio 3D, il pannello “Hierarchy” è adatto ad organizzare i GameObjects all’interno della scena, descrivendo i contenuti della scena corrente tramite relazioni padre-figlio fra i vari GameObjects.

3.2.4 Scripting: La classe MonoBehaviour

Gli scripts sono anche chiamati “behaviours” in Unity. Permettono di rendere gli assets ed i GameObjects interattivi all’interno di una scena. Più script possono essere collegati ad uno stesso GameObject, permettendo un facile riuso di codice.

Unity supporta lo scripting in C#.

La classe **MonoBehaviour** è la classe base da cui deriva ogni script in Unity.

3. BACKGROUND

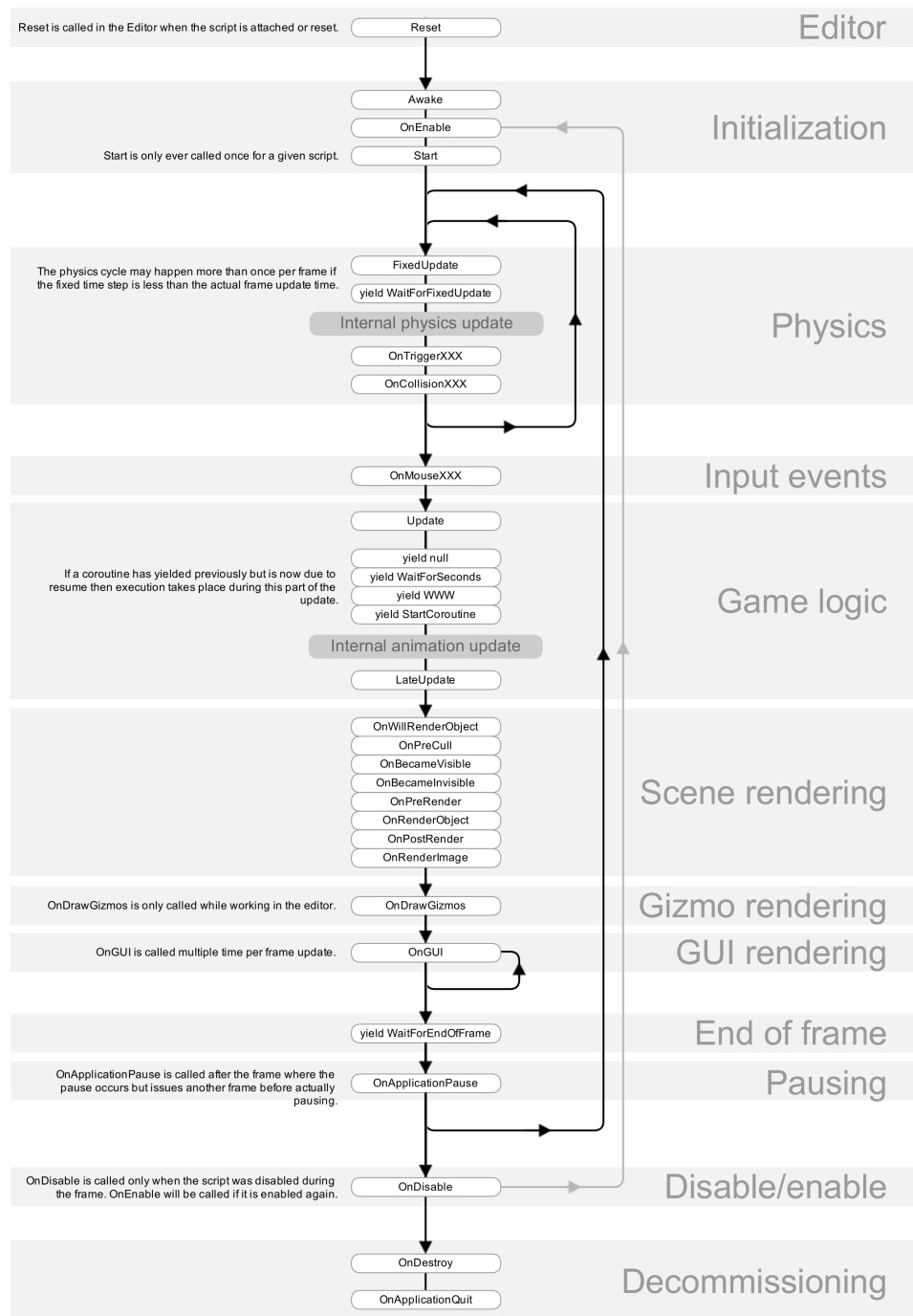


Figura 3.4: Diagramma metodi classe MonoBehaviour [22]

3. BACKGROUND

Quando si crea uno script dal pannello “Project”, lo script eredita in automatico dalla classe **MonoBehaviour**. La classe MonoBehaviour mette a disposizione un framework che permette di collegare gli script a GameObjects all’interno dell’editor, inoltre permette di intercettare determinati eventi relativi al ciclo di vita degli oggetti, in modo da facilitare lo sviluppo delle applicazioni e da eseguire il proprio codice sulla base di quello che sta avvenendo all’interno del progetto.

Di seguito si elencano alcune delle funzioni più importanti della classe **MonoBehaviour**:

- **Awake:** Chiamato quando l’istanza dello script è caricata. E’ principalmente usato per inizializzare variabili o stati, prima che l’applicazione inizi.
- **Start:** Simile al metodo Awake. Viene chiamato ogni volta che un GameObject inizia ad esistere (quando la scena è caricata, oppure quando il GameObject è inizializzato). A differenza del metodo Awake, il metodo Start non può essere chiamato allo stesso frame di Awake se lo script non è stato attivato a tempo di inizializzazione. Questa temporizzazione tra il metodo Awake e il metodo Start risulta essere molto utile in casi in cui esistono due oggetti A e B e l’oggetto A ha bisogno di utilizzare informazioni contenute nell’oggetto B. In questo caso, l’inizializzazione di B dovrebbe essere fatta in Awake, mentre quella di A in Start.
- **Update:** Chiamato ad ogni frame, se lo script MonoBehaviour risulta attivo. Questo è il metodo più usato per implementare funzionalità di scripting di qualsiasi tipo.
- **FixedUpdate:** Simile al metodo Update, ma è indipendente dal Frame-Rate utilizzato dall’ applicazione. Generalmente, viene chiamato più spesso del metodo Update (ma non in ogni caso!).

- **OnDestroy**: chiamato ogni volta che un GameObject viene “distrutto”.
- **StartCoroutine**: Inizia una coroutine. L’esecuzione della coroutine può essere fermata in qualsiasi punto del codice usando lo statement *yield*
- **OnAudioFilterRead**: Unity chiama automaticamente questo metodo ogni volta che un oggetto AudioSource cerca di riprodurre un suono. **OnAudioFilterRead** riceve un array di floats chiamato **data** come parametro, questo array sono i campionamenti PCM che l’oggetto AudioSource sta riproducendo. Il secondo parametro **channels** rappresenta il numero di canali disponibili che stanno riproducendo il suono.

3.2.5 Audio in Unity

Nella vita reale, i suoni sono generati da oggetti e ascoltati attraverso le orecchie delle persone. Il modo in cui il suono è percepito dipende da molti fattori. Una persona che sta ascoltando un suono riesce a percepire approssimativamente da che direzione e da che lontananza il suono proviene. Un oggetto che si muove velocemente emettendo un suono cambierà di tono mentre si muove, come risultato dell’effetto Doppler. Così come l’ambiente all’interno del quale il suono è stato prodotto cambierà il suono stesso, producendo ad esempio eco.

Per simulare l’effetto della posizione del suono, Unity ha bisogno che i suoni siano generati da una componente **AudioSource**, collegata ad un oggetto in una Scena. I suoni emessi vengono poi percepiti da una componente **AudioListener** collegata ad un altro oggetto (di solito alla camera principale della Scena). In questo modo Unity può simulare l’effetto della distanza e della posizione tra l’oggetto Audio Source e l’oggetto Audio Listener e riprodurre il suono di conseguenza.

Unity non riesce a calcolare gli echi e gli effetti puramente partendo dalla Scena, ma questi si possono simulare utilizzando **AudioFilters**.

AudioSource: Un oggetto **Audio Source** permette di riprodurre una clip audio all'interno di una Scena. Questa clip, può essere riprodotta verso un Audio Listener, oppure attraverso un Audio Mixer. Un **Audio Source** può riprodurre qualsiasi tipo di clip audio e può essere configurato per riprodurre le clip in modalita 2D, 3D, o una via di mezzo (SpatialBlend).

AudioListener: Un oggetto **Audio Listener** si comporta come se fosse un microfono all'interno di una Scena. Riceve input da ogni Audio Source nella scena e riproduce i suoni attraverso le casse per computer. Solitamente un Audio Listener viene collegato alla camera principale della scena.

AudioFilters: Un oggetto **Audio Filter** permette di modificare l'output di **Audio Source** o di **Audio Listener**, applicando degli effetti audio. Questi possono permettere di filtrare in suono, oppure di applicare riverbero o altri effetti. Gli effetti sono applicati aggiungendo componenti nell'oggetto contenente l'**Audio Source** o l'**Audio Listener** che sta generando il suono. L'ordine in cui queste componenti vengono collegate agli oggetti è importante, in quanto definirà l'ordine in cui gli effetti andranno a modificare l'audio.

3.3 Gesture Recognition

I metodi di interazione in input tra uomo e macchina, di base, sono mouse e tastiera, che non sempre risultano confacenti agli utenti meno avvezzi all'uso di un computer, o addirittura impossibili per gli utenti portatori di *handicap motori*. È per questo che nel corso degli anni sono stati studiati nuovi metodi alternativi per interagire con un computer.

Agli inizi degli '80 nasceva quindi un primo metodo di input alternativo, lo *Speech Recognition*, che permetteva ad un utente di inserire del testo semplicemente parlando. Col passare degli anni lo Speech Recognition è stato ulteriormente sviluppato, permettendo, nei tempi attuali, di eseguire azioni ben più complesse dell'inserimento di semplice testo, come ad esempio impartire il comando di ricevere informazioni meteo, effettuare acquisti, o

3. BACKGROUND

riprodurre una playlist musicale.

Nel decennio successivo è stato introdotto un nuovo metodo di input in grado di interpretare il linguaggio dei segni, di fatto il precursore del *Gesture Recognition*. Il *Gesture Recognition* non si ferma esclusivamente all'interpretazione e riconoscimento del linguaggio dei segni, ma va oltre, andando ad interpretare gesti e movimenti dell'utente, includendo direzione e velocità dei tali.

Al fine di catturare questi movimenti sono state create diverse tipologie di dispositivi, ognuno con diverse caratteristiche, anche se gran parte di questi sono stati sviluppati per un determinato settore, quello videoludico. La prima azienda videoludica che ha fatto uso di questa innovazione è stata la *Nintendo* con il controller *WiiMote* per la console *Nintendo Wii*, una console molto economica ed accessibile ad un mercato molto ampio, la quale ebbe una diffusione notevole grazie alla novità nel settore.

Dato questo enorme successo, anche altre aziende del settore si cimentarono nel seguire questa nuova modalità di controllo: *Microsoft* sviluppò il sensore *Kinect* per la console *Xbox 360* e successive generazioni, apportando migliorie anche al sensore *Kinect*; la controparte *Sony* sviluppò invece il *Playstation Move*. Il funzionamento di questi 3 controller ha in comune l'utilizzo di telecamere: il *WiiMote* utilizza una telecamera infrarossi sul telecomando, la quale si orienta verso il monitor grazie ad un dispositivo contenente 10 led infrarossi, che funziona da punto cardine; il *Playstation Move* sfrutta una telecamera in grado di riconoscere le sagome umane, ricreando quindi un modello scheletrico di esso, ed usa dei telecomandi specifici per determinare la persona ed i suoi movimenti; il dispositivo *Kinect*, come spiegato nel capitolo precedente, è invece ben più complesso, motivo per il quale ha trovato largo impiego anche al di fuori del mondo videoludico, venendo utilizzato anche in ambito scientifico grazie alla sua elevata efficienza ed il rilascio da parte di Microsoft del *Software Development Kit (SKD)*, che permette lo sviluppo di software da terze parti sfruttando il suddetto dispositivo.

Negli anni a venire sono nati sempre più dispositivi che permettessero il *Gesture*

Recognition, soprattutto nel mondo videoludico ed in particolare per il settore della *Virtual Reality*. I più importanti tra questi sono: *Oculus Rift*, *HTC Vive* e, quello da noi selezionato, il *Leap Motion*.



Figura 3.5: WiiMote [24]



Figura 3.6: Kinect [14]

3.4 Leap Motion

Il nostro sistema fa uso di un controller Leap Motion per il riconoscimento dei movimenti delle mani. Si tratta di un piccolo dispositivo collegabile via USB al PC, che utilizza 2 fotocamere IR monocromatiche, insieme a 3 led infrarossi, per osservare un'area ad una distanza compresa tra 60 e 80 centimetri, con un campo visivo di $140^\circ \times 120^\circ$. Nello specifico, il device è in grado di riconoscere tutte le 10 dita con una precisione fino a 1×10^3 millimetri. Il software del Leap Motion è in grado di riconoscere 27 elementi distinti della mano, incluse ossa e articolazioni, e tracciarle anche nel caso in cui vengano oscurate da altre parti della mano [20]. Il sensore fornisce tre tipi di informazioni spaziali: (1) la posizione di dita e mani; (2) il vettore movimento delle singole dita; (3) la rappresentazione sferica della curvatura della mani. Queste funzionalità permettono all'utente di interagire con il computer attraverso gesture naturali delle mani, come waving, pinching e swiping. L'azienda UltraLeap fornisce delle API in diversi linguaggi di programmazione, e garantisce l'integrazione con Unity3D.

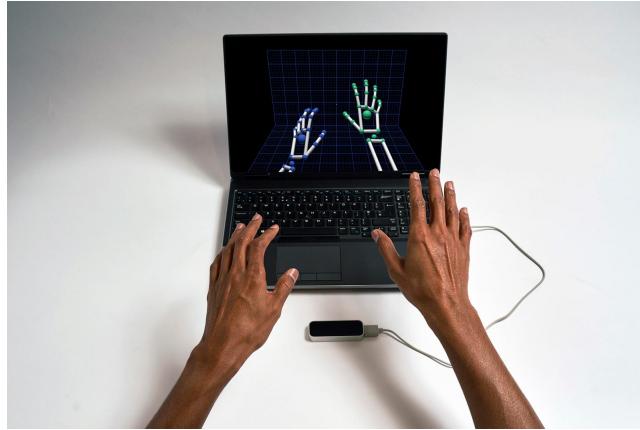


Figura 3.7: Leap Motion Controller [20]

3.5 Fondamenti di Acustica

Il suono viene creato da turbolenze ondulatorie emesse da una “sorgente sonora” e propagate nell’ambiente circostante (solitamente l’aria) [25]. Si tratta di onde di pressione che vengono percepite come suoni dall’orecchio umano.

Le onde sonore possono essere rappresentate graficamente utilizzando un grafico cartesiano, riportante il tempo, (t) sull’asse delle ascisse, e gli spostamenti delle particelle, (s) su quello delle ordinate. Il tracciato esemplifica gli spostamenti delle particelle: alla fine, la particella si sposta dal suo punto di riposo (asse delle ascisse) fino al culmine del movimento oscillatorio, rappresentato dal ramo crescente di parabola che giunge al punto di massimo parabolico. Poi la particella inizia un nuovo spostamento in direzione opposta, passando per il punto di riposo e continuando per inerzia fino ad un nuovo culmine simmetrico al precedente, questo movimento è rappresentato dal ramo decrescente che, intersecando l’asse delle ascisse, prosegue in fase negativa fino al minimo parabolico. Infine, la particella ritorna indietro e ripete nuovamente la sequenza di spostamenti, così come fa il tracciato del grafico.

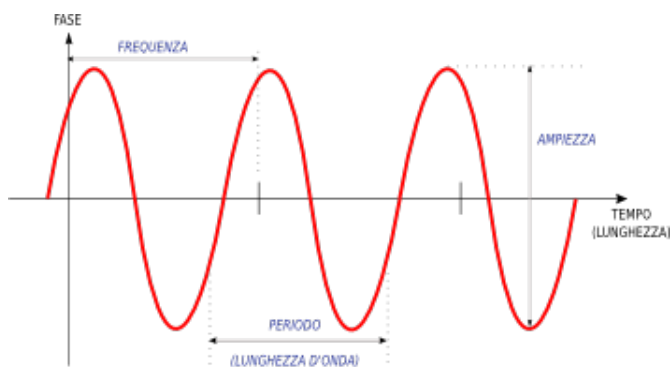


Figura 3.8: Rappresentazione di una Sinusoide

Il periodo (graficamente il segmento tra due creste) è il tempo impiegato dalla particella per tornare nello stesso punto dopo aver cominciato lo spostamento (indica cioè la durata di una oscillazione completa). La distanza dalla cresta all'asse delle ascisse indica, invece, l'ampiezza del movimento. In acustica, si preferisce solitamente usare altre grandezze derivate da queste ultime. Il numero di periodi compiuti in un secondo esprime la frequenza in hertz (Hz). Dall'ampiezza dell'onda, invece, si calcola la pressione sonora, definita come la variazione di pressione rispetto alla condizione di quiete, e la potenza e l'intensità acustica, definita come il rapporto tra la potenza dell'onda e la superficie da essa attraversata; l'intensità delle onde sonore viene comunemente misurata in decibel.

3.6 Digital Audio

Il suono viene creato da turbolenze ondulatorie emesse da una “sorgente sonora” e propagate nell'ambiente circostante (solitamente l'aria). Si tratta di onde di pressione che vengono percepite come suoni dall'orecchio umano. Per convertire un'onda sonora analogica in un segnale digitale, il computer deve essere in grado di misurarne l'ampiezza in determinati punti [26]. Ogni misura rilevata viene chiamata *campione*, perciò la conversione da analogico a formato digitale prende il nome “campionamento del suono”.

Il teorema del campionamento di **Nyquist-Shannon** afferma che, se la frequenza di campionamento è sufficientemente grande, non si hanno perdite

3. BACKGROUND

di informazione rispetto alla forma d'onda originale. Dunque, più ampia sarà la frequenza di campionamento, tanto maggiore sarà anche il numero delle misurazioni d'ampiezza e, quindi, la precisione del segnale digitale.

Se viene usata una **frequenza di campionamento** (*sample rate*) alta, la serie di numeri prodotta contiene pressoché intatta tutta la informazione sulla forma d'onda elettrica analogica iniziale. Nei moderni standard tecnologici, in genere le frequenze di campionamento spaziano dagli 8.000 campioni al secondo (Samples per second, S/s) per la voce telefonica, fino ai 44.100 e più campioni al secondo per la qualità musicale. Queste letture di valori di tensione possono poi cadere in un qualsiasi punto della dinamica del segnale, cioè ogni singolo campione può avere un valore compreso tra il minimo e il massimo possibile.

Siccome potenzialmente si possono avere infiniti valori di lettura di tensione per ogni singolo campione, per completare l'opera di conversione del segnale da analogico in digitale, va suddiviso tutto il possibile range dinamico del segnale in un numero finito di intervalli e ogni singolo intervallo va codificato con un valore digitale ben determinato.

Queste due operazioni si chiamano quantizzazione e codifica di sorgente. La quantizzazione in genere suddivide il range dinamico del segnale in un numero di intervalli potenza del due (2^n *intervalli*), in maniera tale che ogni singolo campione cadrà inevitabilmente in uno degli intervalli quantizzati e potrà così essere codificato digitalmente con n bit. I valori più ricorrenti di digitalizzazione attualmente usati vanno da un minimo di 8 bit per campione in campo telefonico (range dinamico del segnale suddiviso in 256 intervalli), fino a 20 e più bit per campione (range dinamico del segnale suddiviso in un milione e più di intervalli).

3. BACKGROUND

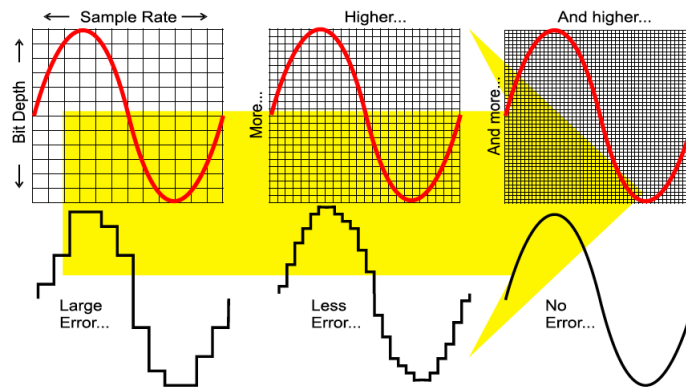


Figura 3.9: Margine di errore per frequenze di campionamento crescenti [27]

Frequenza di Campionamento. La **frequenza di campionamento** è il numero di misurazioni fatte al secondo per convertire un segnale analogico in digitale. Aumentando le misurazioni al secondo, siamo in grado di raccogliere più informazioni reali, e dunque di ricostruire un'immagine del suono molto accurata.

Profondità di Bit. La **profondità di bit** permette di definire il range dinamico del segnale. A ogni campionamento è assegnato un'ampiezza specifica. Una **profondità di bit** elevata, permetterà di avere più spazio tra volumi alti e volumi bassi, migliorando così il range dinamico dei campionamenti e della registrazione. In generale, per ogni bit che si aggiunge alla **profondità di bit** utilizzata, il range dinamico aumenta di 6dB.

Codifica Digitale PCM. La **codifica digitale PCM** (abbreviazione che sta per Pulse Code Modulation) è il nome generico per il sistema di registrazione digitale. E' la forma standard in cui viene trattato l'audio digitale in computer, CD, telefonia digitale e altre applicazioni di audio digitale. Nel testo, useremo il termine “**codifiche PCM**” per riferirci ad una sequenza di campionamenti digitali che descrivono un segnale analogico.

3.7 Artificial Neural Networks

Le *Reti Neurali Artificiali* (*Artificial Neural Networks*, o *ANN*) sono modelli computazionali ispirati alle reti neurali biologiche, come quelle che compongono il cervello, e di conseguenza sono costituiti da un insieme di interconnessioni di informazioni formate tra neuroni artificiali che elaborano dati e li scambiano l'un l'altro, esattamente come le sinapsi. Ogni neurone è connesso con molti altri ed i collegamenti possono essere di tipo rafforzativo o inibitorio rispetto all'attivazione degli altri neuroni ad esso connessi. Ognuno di questi neuroni usa una funzione apposita per combinare tutti i valori di input con una *funzione di attivazione*, che elabora e restituisce l'output del neurone:

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

Dove w_i è la matrice dei pesi (***weights***) assegnati a ciascun input, che viene aggiunta ad un termine b (***bias***). I due insiemi ***weights*** e ***bias*** vengono generati nella fase di addestramento e saranno conservati per l'utilizzo. Tramite questa funzione, la rete neurale filtra gli input, operando solo quelli che rientrano in un determinato limite imposto. Una rete neurale è spesso di tipo *adattivo*, tale da mutare la propria struttura in base a modelli ricevuti in input durante la *fase di apprendimento*.

Per ottimizzare la funzione di attivazione, è possibile introdurre una soglia che servirà a decrementare il valore d'ingresso della funzione. Il metodo più diffuso è l'utilizzo di una delle seguenti funzioni:

- Funzione di **Sigmund**;
- Funzione di **TanH** (tangente iperbolica);
- Funzione di **ReLU** (rectified linear unit);

3. BACKGROUND

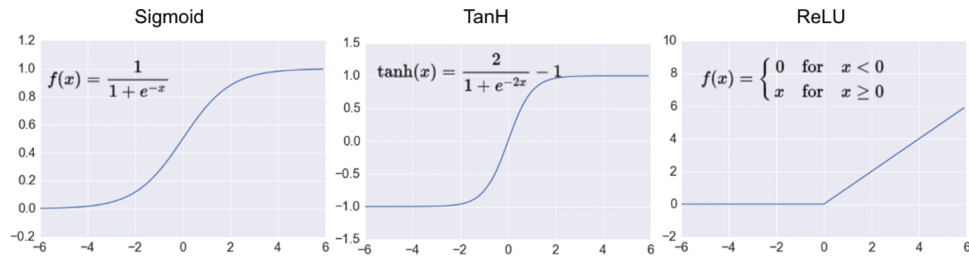


Figura 3.10: [28]

La struttura di una rete neurale può essere divisa in tre livelli, ognuno dei quali ha un numero distinto di neuroni:

- Livello di input;
- Livello nascosto;
- Livello di output;

Uno dei difetti delle reti neurali, nonostante l'elevata efficienza, è la non spiegabilità delle decisioni avvenute al loro interno, proprio per questo, questi modelli sono chiamati “*Black Box*”.

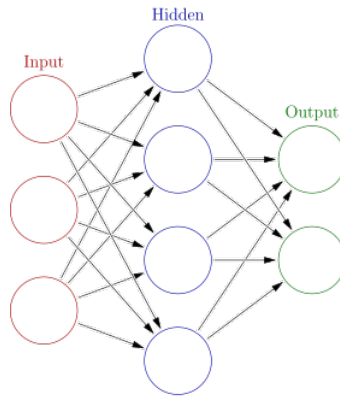


Figura 3.11: Rete Neurale Artificiale [29]

3.8 Paradigmi di apprendimento

L'apprendimento può essere effettuato seguendo tre tipi diversi di paradigmi: *apprendimento supervisionato*, *apprendimento non supervisionato* ed *apprendimento per rinforzo*.

- **Apprendimento supervisionato (*supervised learning*):**

Quando si ha a disposizione un insieme di dati, anche chiamato *training set*, contenente esempi d'ingresso con il relativo output, la rete imparerà a stabilire la relazione che collega tutti i dati del *training set*.

Nella fase di addestramento, vengono utilizzati appositi algoritmi come il **backpropagation**, che ciclicamente sfrutta i dati ricevuti in input per stimare i *weight* ed i *bias* della rete di neuroni in modo tale da ridurre al minimo l'errore di riconoscimento del *training set*. Se l'addestramento è stato effettuato con successo, quindi con una *accuracy* elevata, la rete ha imparato a riconoscere la relazione che lega i dati in ingresso con quelli in uscita e sarà quindi in grado di effettuare previsioni anche su dati differenti da quelli del *training set*, ovvero su qualsiasi dato (purché rispetti la forma di input accettata) ricevuto in ingresso. La rete sarà quindi in grado di risolvere problemi di classificazione e potrà essere chiamata **classifier**.

- **Apprendimento non supervisionato (*unsupervised learning*):**

Metodo di apprendimento basato su algoritmi che riclassificano e organizzano i dati passati in input in base a caratteristiche comuni tra di loro, per effettuare previsioni sugli input ricevuti successivamente. A differenza dell'apprendimento supervisionato, i dati ricevuti in fase di addestramento non sono annotati, dato che le classi da riconoscere non sono conosciute inizialmente, ma verranno distinte durante l'addestramento stesso.

- **Apprendimento per rinforzo (*reinforcement learning*):**

Questo metodo di apprendimento, a differenza dei due precedenti, opera come un automa deterministico, occupandosi di problemi di decisioni sequenziali, dove l'output dipende esclusivamente dallo stato in cui si trova il sistema determinandone il successivo. La qualità delle decisioni dipende da un valore chiamato *ricompensa*, maggiore è la ricompensa e maggiore sarà la propensione del sistema a prendere quella decisione.

3. BACKGROUND

Le *ricompense* possono essere sia positive che negative, in modo tale da incoraggiare o scoraggiare determinate scelte.

La scelta del paradigma dipende dal tipo di ecosistema che si vuole creare e dal tipo di dati di cui si è in possesso. Se si dispone di dati da fornire in input, e si conosce l'output atteso (detto **label**), la scelta ricade automaticamente sull'*apprendimento supervisionato*.

CAPITOLO 4

MARCOSMILES 2.0

L'obiettivo principale di MarcoSmiles 2.0 è quello di superare i limiti della sua precedente version: MarcoSmiles [7]. L'idea è quella di permettere all'utente di poter realizzare delle performance musicali in maniera più naturale e performante. Il sistema utilizza esclusivamente informazioni contenute nella configurazione delle mani, e associa tali configurazioni a note musicali in real-time. È importante ricordare che il sistema non dispone di una configurazione predefinita. Sarà compito dell'utente creare l'astrazione dello strumento addestrandolo a riconoscere configurazioni delle mani secondo le sue preferenze. Nella sua versione precedente, il lavoro è stato svolto interamente utilizzando il linguaggio di programmazione Java. Durante lo sviluppo di questo lavoro, tuttavia, sono emerse diverse diverse problematiche in merito alla reattività dell'interfaccia, e sulle performance in generale, imputabili alla scelta di Java come linguaggio su cui basare il sistema. Inoltre anche la gestione della generazione del suono real-time è risultata particolarmente difficoltosa, motivo per cui, il progetto è stato accantonato. Sulla base di queste esperienze abbiamo deciso di riprogettare il sistema. La scelta sulle tecnologie da usare è ricaduta su Unity3D e C# per quanto riguarda l'interfaccia grafica e le funzioni di base, mentre per la parte di Machine Learning, abbiamo usato

la libreria SciKit Learn in Python. Il sistema da noi sviluppato è dunque strutturato in una componente hardware, e una componente software. La componente hardware è rappresentata dal Leap Motion Controller, che può essere posizionato a piacimento dell'utente, solitamente su una scrivania con i sensori rivolti verso l'alto. La componente software consiste nel sistema che effettua l'associazione configurazione - nota musicale in real-time. In fase di progettazione abbiamo suddiviso la parte software in 3 moduli:

1. **Modulo Unity:** Interfacciamento Leap Motion Controller e GUI;
2. **Modulo Machine Learning:** Registrazione dati, elaborazione e testing
3. **Modulo Sonoro:** Generazione audio real-time & Pulse-Code Modulation

Il lavoro di tesi in oggetto riguarda il primo modulo del sistema. In particolare verranno descritte le principali funzionalità del sistema, la costruzione e la progettazione dell'interfaccia grafica e del dataset, le features selezionate per il machine learning, e la gestione delle configurazioni del sistema.

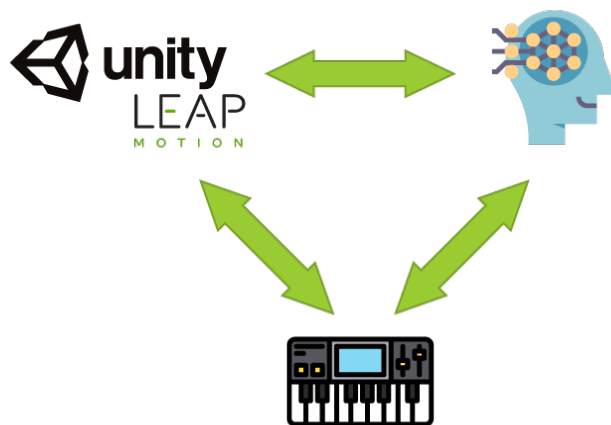


Figura 4.1: Diagramma Interfacciamento Moduli

CAPITOLO 5

MODULO UNITY

5.1 Data Representation

La scelta dei dati da utilizzare nel dataset è di fondamentale importanza nell’ambito di un problema di classificazione, infatti l’output della ANN dipende in gran parte dalla scelta effettuata. Per ogni sample catturato dal LeapMotion bisogna considerare un adeguato insieme di features da utilizzare per la gesture recognition. Una soluzione consisterebbe nell’utilizzare i “raw data” acquisiti dal Leap Motion come insieme di features. Tuttavia questo approccio non permette di ottenere buoni risultati visto che i dati sono dipendenti da numerosi fattori quali la posizione, l’orientamento e la dimensione delle mani. Dunque anche un piccolo movimento produrrebbe una minor precisione in fase di classificazione. Nell’effettuare la scelta dei dati da utilizzare abbiamo considerato lo studio effettuato nella precedente versione del sistema MarcoSmiles. In questo tipo di problema la letteratura suggerisce di adottare un insieme di features indipendente dai suddetti fattori. Utilizzando questo approccio, una gesture è tale indipendentemente dalla traslazione, rotazione e dimensione della mano. Ciò significa che il sistema è in grado di riconoscere la gesture senza tener conto dei fattori sopracitati. Le API fornite

dal Leap Motion permettono di ottenere e manipolare diverse informazioni circa le mani. Denotiamo con l_5, l_4, l_3, l_2, l_1 le dita della mano sinistra da mignolo a pollice, mentre denotiamo con r_1, r_2, r_3, r_4, r_5 le dita della mano destra, da pollice a mignolo. Inoltre indichiamo con $S_{LH} = \{l_5, l_4, l_3, l_2, l_1\}$ l'insieme delle dita della mano sinistra, e con $S_{RH} = \{r_1, r_2, r_3, r_4, r_5\}$ l'insieme delle dita della mano destra. Le feature che abbiamo scelto di utilizzare sono le seguenti:

- **Finger Flexion (FF)**: sia $f \in S_{LH} \cup S_{RH}$ un dito, la *flessione* di f è l'angolo θ tra i vettori direzione associati al metacarpo ed alle ossa distali di f (vedi Figura).
- **Nearest Finger Angle (NFA)**: siano f_1, f_2 due dita tali che $f_1, f_2 \in S_{LH}$ o $f_1, f_2 \in S_{RH}$, l'NFA è l'angolo θ tra f_1 ed f_2 ed è calcolato come l'angolo tra il vettore-direzione di f_1 ed il vettore-direzione di f_2 (vedi Figura).

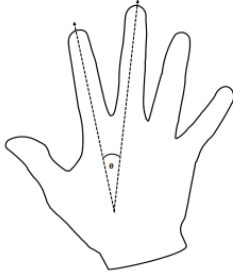


Figura 5.1: Nearest Fingers Angles (NFA)

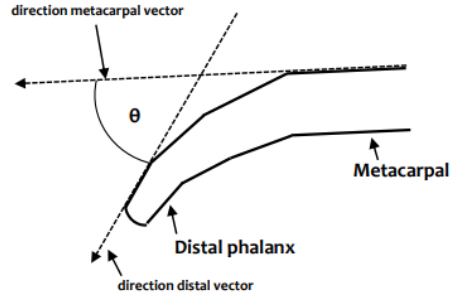


Figura 5.2: Finger Flexion (FF)

5.2 Dataset

La struttura del nostro dataset sarà la seguente:

$$(C_{LH}, C_{RH}, id)$$

Dove C_{LH} e C_{RH} sono configurazioni di dati relativi alla posizione della mano sinistra e destra, mentre id sarà la *label* corrispondente alla nota musicale che

si desidera suonare.

Ogni configurazione delle mani è composta da diverse combinazioni delle *features* da noi selezionate.

Le combinazioni di *features* presenti nelle configurazioni C_{LH} e C_{RH} sono le seguenti:

- C_{LH} :

$FF_0(l_1), FF_1(l_2), FF_2(l_3), FF_3(l_4), FF_4(l_5),$

$NFA_0(l_0, l_1), NFA_1(l_1, l_2), NFA_2(l_2, l_3), NFA_3(l_3, l_4)$

- C_{RH} :

$FF_0(r_1), FF_1(r_2), FF_2(r_3), FF_3(r_4), FF_4(r_5),$

$NFA_0(r_0, r_1), NFA_1(r_1, r_2), NFA_2(r_2, r_3), NFA_3(r_3, r_4)$

Nei *training set* utilizzati per l'addestramento ad ogni nota corrispondono 500 combinazioni di (C_{LH}, C_{RH}, id) , dove l'elemento *id* è comune a tutte le 500. Ognuna di queste combinazioni varierà leggermente dalle altre, dato che la registrazione del dataset è stata effettuata salvando la posizione delle mani nell'arco di due minuti, durante i quali si presentano leggere variazioni, volontarie e non. Queste variazioni volontarie sono effettuate al fine di fornire più informazioni all'algoritmo di apprendimento, dato che in una situazione reale la possibilità di riposizionare le mani nella esatta posizione di addestramento è scarsa.

5.3 Gestione Progetto Unity

Il codice del progetto è ripartito in moduli, in modo da separare le responsabilità assegnate a ciascuna funzione del sistema:

- **GameMaster:**

La classe `_GM`, contenuta all'interno del file `_GM.cs` rappresenta il "Game Master" del nostro sistema. E' una classe *MonoBehaviour* che mantiene aggiornato lo stato dell'applicazione. La classe viene utilizzata

per mantenere la referenza alle variabili globali e agli oggetti istanziati di cui vogliamo tener traccia durante l'esecuzione dell'applicazione. Rappresenta dunque il processo principale, attraverso il quale sono gestite la navigazione tra le scene, il salvataggio dei dati, e tutte le funzioni principali del sistema.

- **DatasetHandler:**

Lo script `DatasetHandler.cs` contiene i metodi che utilizziamo per calcolare le features per ogni mano, che vengono utilizzate dal classificatore per effettuare la predizione.

- **DataToStore:**

All'interno dello script `DataToStore.cs` è definita l'omonima classe `DataToStore`. Rappresenta l'oggetto che incapsula tutte le features relative alla configurazione di una singola mano.

- **FileUtils:**

In questo script sono presenti metodi relativi alla gestione dei dataset e delle configurazioni del sistema. In particolare vengono gestite le operazioni di salvataggio delle posizioni nel dataset, e della relativa cancellazione. È anche presente il metodo che consente l'interfacciamento tra lo script di Machine Learning e C#.

- **MS_LeapController e MS_LeapListener:**

In questi script sono gestite la connessione del Leap Motion Controller al sistema, e il suo interfacciamento con Unity3D.

- **PanelUtils:**

All'interno dello script `PanelUtils.cs` sono presenti i metodi che gestiscono le operazioni per cambiare, importare ed esportare una configurazione del sistema.

- **Position:**

La classe `Position`, locata all'interno dell'omonimo script, incapsula

una configurazione di entrambe le mani, identificata attraverso un id numerico.

- **StartLearn:**

Questo script contiene il metodo per lanciare lo script di Machine Learning quando l'utente desidera addestrare il sistema.

- **TrainingScript:**

Lo script `TrainingScript.cs` si occupa di gestire la fase in cui l'utente vuole registrare una configurazione delle mani.

- **TestML:**

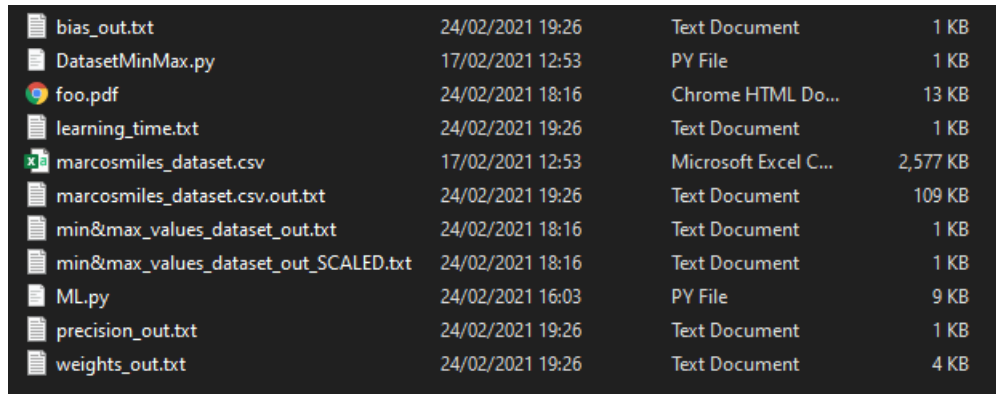
Lo script `TestML.cs` contiene diversi metodi inerenti al *Machine Learning*. Ha il compito leggere i file necessari all'utilizzo della rete neurale, e di effettuare la previsione della nota da suonare. Inoltre è presente il metodo per lo *scaling* dei dati.

Nella cartella `AudioFiles` sono presenti tutti gli script inerenti alla generazione del suono real-time.

5.4 Gestione Salvataggio dati

5.4.1 Gestione Directory

I file utilizzati dal sistema `MarcoSmiles` risiedono all'interno della directory `C:\Users\Utente\AppData\Local\Low\DefaultCompany\MarcoSmiles`. Questo path non è statico, bensì cambia in base al dispositivo utilizzato, e solitamente per accedervi bisogna semplicemente sostituire `Utente` con il nome utente in uso sul dispositivo. Utilizziamo il termine “configurazione del sistema” per riferirci all'insieme dei seguenti file:



bias_out.txt	24/02/2021 19:26	Text Document	1 KB
DatasetMinMax.py	17/02/2021 12:53	PY File	1 KB
foo.pdf	24/02/2021 18:16	Chrome HTML Do...	13 KB
learning_time.txt	24/02/2021 19:26	Text Document	1 KB
marcosmiles_dataset.csv	17/02/2021 12:53	Microsoft Excel C...	2,577 KB
marcosmiles_dataset.csv.out.txt	24/02/2021 19:26	Text Document	109 KB
min&max_values_dataset_out.txt	24/02/2021 18:16	Text Document	1 KB
min&max_values_dataset_out_SCALED.txt	24/02/2021 18:16	Text Document	1 KB
ML.py	24/02/2021 16:03	PY File	9 KB
precision_out.txt	24/02/2021 19:26	Text Document	1 KB
weights_out.txt	24/02/2021 19:26	Text Document	4 KB

Figura 5.3: Configurazione di Sistema

che rapprestano le posizioni scelte dall'utente (`dataset.csv`) e la rete neruale addestrata a riconoscerle. L'utente dispone della possibilità di importare ed esportare configurazioni del sistema attraverso una sezione dedicata all'interno dell'interfaccia grafica. Le configurazioni vengono importate direttamente all'interno del path `C:\Users\Utente\AppData\Local\Low\DefaultCompany\MarcoSmiles\MyDataset`, per poi essere utilizzate selezionandole attraverso la relativa opzione all'interno della GUI.

5.4.2 Registrazione e Salvataggio Configurazioni Mani

In fase di training, l'utente registra le configurazioni delle mani da associare alla nota desiderata. Verranno effettuate 500 misurazioni, corrispondenti a 500 righe nel dataset. Ques'ultimo è posizionato all'interno del path menzionato nella sezione precedente. Abbiamo implementato l'intero processo utilizzando il design pattern coroutine. I metodi descritti sono riportati all'interno del file `TrainingScript.cs`. Quando l'utente desidera far partire la registrazione di una configurazione, viene invocato il metodo `Trainer()` che farà partire la coroutine:

Listing 5.1: Metodo `Trainer()`

```
1 public void Trainer()  
2 {
```

```
3         if(!counting_flag)
4         {
5             count = COUNT_DEF + 1;
6             record_count = RECORD_COUNT_DEF;
7
8             StartCoroutine(Waiter());
9             counting_flag = true;
10        }
11    }
```

Il metodo invocato da `Trainer()` è `Waiter()`; si occuperà di avviare il countdown per la registrazione, e al suo termine darà inizio alla registrazione delle posizioni:

Listing 5.2: Metodo `Waiter()`

```
1  IEnumerator Waiter()
2  {
3      if (count > 0)
4      {
5          // decrementa di 1 il numero di secondi da attendere
6          prima della registrazione e lo mostra a schermo
7          count--;
8          countDown_Text.text = count.ToString();
9          position_Text.text = text1;
10         Debug.Log($"COUNTDOWN: {count.ToString()}");
11
12
13         // effettua una pausa di 1 secondo
14         yield return new WaitForSeconds(1);
15
16         // inizia la coroutine per il prossimo secondo da scalare
17         StartCoroutine(Waiter());
18     }
19     else
20     {
21         // countdown terminato
22         counting_flag = false;
23     }
```

5. MODULO UNITY

```
24         // inizia la coroutine per la prima posizione da salvare
25         StartCoroutine(WaiterRecording());
26     }
27 }
```

Il metodo `WaiterRecording()` gestisce la Coroutine che registra la posizione assunta attraverso il contatore `record_count`:

Listing 5.3: Metodo `WaiterRecording()`

```
1  IEnumerator WaiterRecording()
2  {
3      if (record_count > 0)
4      {
5          // decrementa di 1 le posizioni totali rimanenti da
6          //registrare e mostra a schermo
7          record_count--;
8          recording_Text.text = record_count.ToString();
9          position_Text.text = text2;
10
11         // effettua una pausa di 125 millisecondi
12         yield return new WaitForSeconds(0.125f);
13
14         // aggiunge la posizione corrente in lista
15         DataSelector();
16
17         // inizia una nuova coroutine per la prossima
18         // posizione da salvare
19         StartCoroutine(WaiterRecording());
20     }
21     else
22     {
23         // registrazione terminata
24         recording_flag = false;
25
26         // salva su file le posizioni registrate
27         FileUtils.Save(_GM.list_posizioni);
28     }
29 }
```

30 }

La corotuine precedente fa uso del metodo `DataSelector()` per creare una lista con tutte le configurazioni delle mani registrate;

Listing 5.4: Metodo `DataSelector()`

```
1
2 private void DataSelector()
3 {
4     // mano sinistra
5     var left_hand = new DataToStore(
6         _GM.hand_L,
7         DatasetHandler.getFF(_GM.hand_L.Fingers[0], true),
8         DatasetHandler.getFF(_GM.hand_L.Fingers[1]),
9         DatasetHandler.getFF(_GM.hand_L.Fingers[2]),
10        DatasetHandler.getFF(_GM.hand_L.Fingers[3]),
11        DatasetHandler.getFF(_GM.hand_L.Fingers[4]),
12
13        DatasetHandler.getNFA(_GM.hand_L.Fingers[0], _GM.hand_L.Fingers[1]),
14        DatasetHandler.getNFA(_GM.hand_L.Fingers[1], _GM.hand_L.Fingers[2]),
15        DatasetHandler.getNFA(_GM.hand_L.Fingers[2], _GM.hand_L.Fingers[3]),
16        DatasetHandler.getNFA(_GM.hand_L.Fingers[3], _GM.hand_L.Fingers[4]));
17
18
19     // mano destra
20     var right_hand = new DataToStore(
21         _GM.hand_R,
22         DatasetHandler.getFF(_GM.hand_R.Fingers[0], true),
23         DatasetHandler.getFF(_GM.hand_R.Fingers[1]),
24         DatasetHandler.getFF(_GM.hand_R.Fingers[2]),
25         DatasetHandler.getFF(_GM.hand_R.Fingers[3]),
26         DatasetHandler.getFF(_GM.hand_R.Fingers[4]),
27
28         DatasetHandler.getNFA(_GM.hand_R.Fingers[0], _GM.hand_R.Fingers[1]),
29         DatasetHandler.getNFA(_GM.hand_R.Fingers[1], _GM.hand_R.Fingers[2]),
30         DatasetHandler.getNFA(_GM.hand_R.Fingers[2], _GM.hand_R.Fingers[3]),
31         DatasetHandler.getNFA(_GM.hand_R.Fingers[3], _GM.hand_R.Fingers[4]));
32
```

```
33 //aggiunge la posizione registrata alla lista delle posizioni da salvare
34 _GM.list_posizioni.Add(new Position(left_hand: left_hand,
35 right_hand: right_hand, id: currentNoteId));
36 }
37
```

infine, all'interno di `WaiterRecording()` viene invocato, dal file `FileUtils.cs`, il metodo `Save(List<Position> data)`; si occuperà di salvare i dati nella lista di posizioni all'interno del dataset. In primo luogo controlla se il file `.csv` del dataset è presente nella cartella `AppData` dell'utente, e se non lo è lo crea.

Listing 5.5: Sezione del Metodo `Save()`

```
1 public static void Save(List<Position> data)
2 {
3     // controlla se il file nel path esiste
4     if(File.Exists(GeneratePath(filename)))
5     {
6         // se il file presente nel path, crea una stringa
7         // contenente tutte le features e id della nota, per
8         // salvare sul file nel path
9
10        foreach (var item in data)
11        {
12            ...
13
14            // va a capo per la prossima feature da salvare
15            str += Environment.NewLine;
16
17            // stampa la stringa str sul file, aggiungendo quindi
18            // tutte le posizioni registrate per la nota selezionata
19            File.AppendAllText(GeneratePath(filename), str);
20        }
21
22        // pulisce la lista delle posizioni, per poter registrare
23        // altre posizioni nella stessa sessione, senza dover
24        // riavviare il programma

```

```
25         _GM.list_posizioni.Clear();
26
27         // stampa il messaggio nella console di debug
28         Debug.Log("DATASET SAVED");
29     }
30     else
31     {
32         // se il file non presente nel path, lo crea
33
34         File.Create(GeneratePath(filename)).Dispose();
35
36         // effettua una chiamata ricorsiva per salvare sul file
37         // appena creato
38         Save(data);
39
40         // stampa il messaggio nella console di debug
41         Debug.Log("FILE CREATO");
42     }
43 }
```

5.5 Interfacciamento Python e Unity

Un altro importante problema affrontato è stato l'interfacciamento del linguaggio di programmazione Python, utilizzato all'interno dello script di Machine Learning, con Unity, che non lo supporta nativamente. Per realizzare un sistema personalizzabile, l'utente deve poter addestrare il sistema utilizzando configurazioni delle mani di suo gradimento. Questo comporta la necessità di lanciare lo script python per il Machine Learning ogni volta che l'utente voglia addestrare il sistema. Quando si effettua la build di un applicazione in Unity, vengono creati, oltre all'eseguibile, delle directory contenenti i file di cui necessita l'applicazione per il suo funzionamento.

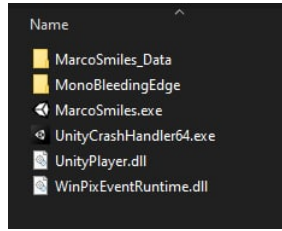


Figura 5.4: Files creati nel processo di build

Nelle directory è presente anche una copia di tutti i file contenuti nella cartella “Resources” appartenente al progetto dell’applicazione, utilizzata da Unity come “target folder”, in cui posizionare tutti i file che si desidera includere nella build. In fase di testing, abbiamo eseguito una build del sistema, tuttavia lo script `ML.py` posto all’interno della directory “Resources”, non è risultato presente, a causa dell’estensione `.py`. Sono state analizzate diverse opzioni per trovare una risoluzione al problema; la prima è “Python for Unity” [31], un package in fase di sviluppo, creato dal team di Unity. La sua funzione è quella di permettere di accedere alle API che Unity fornisce in C# attraverso Python, tuttavia il suo sviluppo è ancora in beta. La versione di Python che supporta è la 2.7, inoltre, il suo funzionamento è garantito esclusivamente all’interno dell’editor Unity, e non nelle build standalone. Considerato che il nostro script per il Machine Learning utilizza Python 3.7, e la nostra necessità di creare una build stand-alone, abbiamo deciso di non utilizzare “Python for Unity”. La seconda opzione presa in considerazione è stata il package di terze parti “IronPython” [32]. Iron Python è un’implementazione open-source del linguaggio Python che supporta una stretta integrazione con il Framework .NET [33]. Il linguaggio supporta sia librerie Python che librerie .NET. Purtroppo anche questo package supporta solo Python 2.7, ragion per cui abbiamo dovuto scartarlo data l’incompatibilità con lo script per il Machine Learning, oltre al fatto che introdurre un software di terze parti non riconosciuto da Unity sarebbe stato rischioso. Non essendo riusciti a trovare una soluzione che facesse al caso nostro, abbiamo provato a realizzarne una ad-hoc, eseguendo diversi test in fase di build dell’applicazione. In particolare abbiamo notato che trasformando il file python dello script per il Machine Learning in un

semplice file di testo con estensione `.txt`, quest'ultimo veniva "trasportato" con successo nelle directory della build, a differenza del file con estensione `.py`. Sulla base di questa osservazione abbiamo costruito la soluzione per risolvere il problema riscontrato. L'idea consiste nel creare, all'avvio dell'applicazione, il file `ML.py` richiesto per il Machine Learning, copiando il contenuto del file `ML.txt` presente nelle directory della build. All'interno del metodo `Awake()`, invocato quando l'applicazione viene aperta,

Listing 5.6: Sezione del Metodo `Awake()`

```
1 private void Awake()
2 {
3     string nameFile = "ML";
4     var MLFile = Resources.Load<TextAsset>("Text/" + nameFile);
5     FileUtils.SavePy(MLFile.bytes, MLFile.name);
```

viene chiamato il metodo `SavePy()`, che si occuperà di copiare il file `ML.txt`, e convertirlo in file `.py`, e inserirlo all'interno del path `C:\Users\Utente\AppData\Local\Low\DefaultCompany\MarcoSmiles`.

Listing 5.7: Metodo `SavePy()`

```
1 public static void SavePy(byte[] file, string name)
2 {
3     // imposta nome ed estensione al file da salvare
4     string filename = name + ".py";
5     string filePath = $"{path}/{folderName}/{filename}";
6     try
7     {
8         // se il file non presente nel path, lo crea
9         if (!File.Exists(filePath))
10        {
11            using (var fs = new FileStream(filePath,
12                FileMode.Create, FileAccess.Write))
13            {
14                fs.Write(file, 0, file.Length);
15            }
16        }
17        else
```

```
18         {
19             // se il file presente nel path, stampa il
20             // messaggio nella console di debug
21             Debug.Log("FILE GIA ESISTENTE");
22         }
23     }
24     catch (Exception ex)
25     {
26         Debug.Log("Exception caught in process:" + ex.ToString());
27     }
28 }
```

Il path non è statico e dipende dal dispositivo utilizzato, ed è facilmente accessibile grazie all'attributo `Application.persistentDataPath` presente nelle API di Unity.

Listing 5.8: Accesso al Path utilizzato

```
1 static string path = Application.persistentDataPath;
```

5.6 GUI

L'interfaccia grafica è stata realizzata interamente in Unity3D. Nel complesso l'interfaccia è composta da 3 scene:

- (1) *Main Menu Scene*
- (2) *Training Scene*
- (3) *Play Scene*

La scena *Main Menu* (1) è presente il logo del sistema MarcoSmiles, e fornisce accesso alle restanti due attraverso dei semplici pulsanti. In basso a destra è presente una sezione in cui è possibile accedere a diverse funzioni relative alle configurazioni del sistema. Nello specifico è possibile cambiare, importare ed esportare configurazioni di cui l'utente dispone, attraverso dei bottoni. Infine è presente un tasto *Quit* per chiudere l'applicazione.



Figura 5.5: Main Menu Scene

Nella scena di *Training* (2) l'utente può registrare le configurazioni delle mani che vuole associare a delle specifiche note. Al centro è possibile visualizzare in real-time la proiezione delle mani che rileva il Leap Motion, mentre in basso è posto un Piano virtuale la cui funzione è quella di poter permettere all'utente di scegliere la nota da allenare. I tasti del Piano sono tutti selezionabili con un click, e nel momento in cui ciò avviene, il tasto viene evidenziato in rosso in modo da comunicare visivamente all'utente qual è la nota scelta. Accanto al piano è posto un pulsante *Info*, da cui è possibile accedere ad un PopUp contenente informazioni riguardo ai pulsanti presenti nella scena e in merito al processo di training. In basso a sinistra sono presenti due pulsanti che permettono rispettivamente di poter tornare nel menu principale, e di eliminare tutte le configurazioni delle mani registrate all'interno della configurazione del sistema attuale.

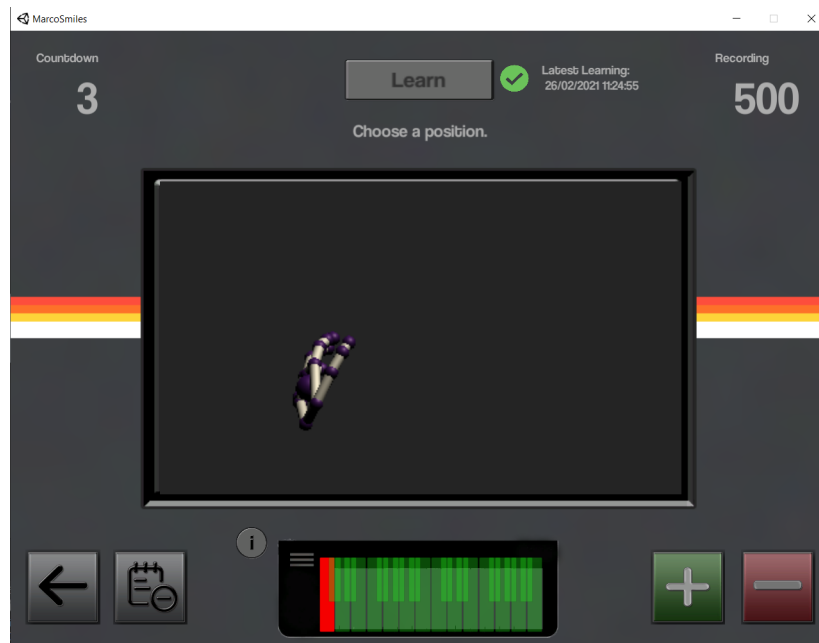


Figura 5.6: Training Scene

Il processo di training si articola in più fasi; l'utente seleziona la nota da allenare dalla tastiera, per poi cliccare il pulsante verde $+$. Appena viene premuto il pulsante, apparirà in alto un timer di 3 secondi. In questo lasso di tempo l'utente deve assumere la configurazione delle mani che desidera associare alla nota selezionata. Al termine del timer partirà la registrazione delle posizioni. Verranno effettuate 500 misurazioni, corrispondenti a 500 righe nel dataset, per un totale di circa 3 minuti di addestramento. Lo stato del processo di registrazione è indicato dal relativo contatore in alto a destra che indica il numero di registrazioni mancanti. Una volta terminato il processo, la nota selezionata verrà evidenziata in verde per segnalare all'utente che è stata allenata. È possibile anche allenare la stessa nota più volte. Se l'utente non è soddisfatto della configurazione registrata può eliminarla attraverso il tasto rosso $-$, o semplicemente proseguire con la registrazione di altre note. Cliccando il tasto rosso verrà eliminata soltanto l'ultima configurazione delle mani registrata, corrispondente alle ultime 500 righe del dataset. L'utente dispone inoltre della possibilità di eliminare tutte le configurazioni registrate attraverso il relativo tasto. Questa operazione porterà alla cancellazione di tutte le righe del dataset. Quando l'utente avrà registrato le posizioni desiderate,

potrà procedere con la creazione di una configurazione del sistema allenato premendo sul tasto *Learn*. Questa operazione manderà in esecuzione lo script python per il Machine Learning. Lo script richiederà tempo proporzionale alle risorse computazionali della macchina sul quale è in esecuzione. Al termine dell'esecuzione l'utente potrà testare la configurazione del sistema creata accedendo alla scena *Play* (3), tornando nel menu principale. Accanto al tasto *Learn* è inoltre indicato attraverso una spunta se è stato eseguito il learning con successo, e l'orario in cui è stata terminata l'esecuzione dell'ultimo addestramento.

Nella scena *Play* (3) l'utente può eseguire performance musicali utilizzando la configurazione del sistema selezionata. Al centro, come nella scena di *Training* (2), sono visualizzate real-time le mani dell'utente. In basso è posto un Piano virtuale, le cui note vengono evidenziate in rosso quando l'utente assume una configurazione delle mani che il sistema è in grado di riconoscere. Il Piano è dotato anche di due pulsanti che permettono di cambiare il range di ottava selezionato. L'opzione predefinita prevede l'utilizzo delle ottave C3-C4, e i pulsanti permettono rispettivamente aumentare e diminuire il range.

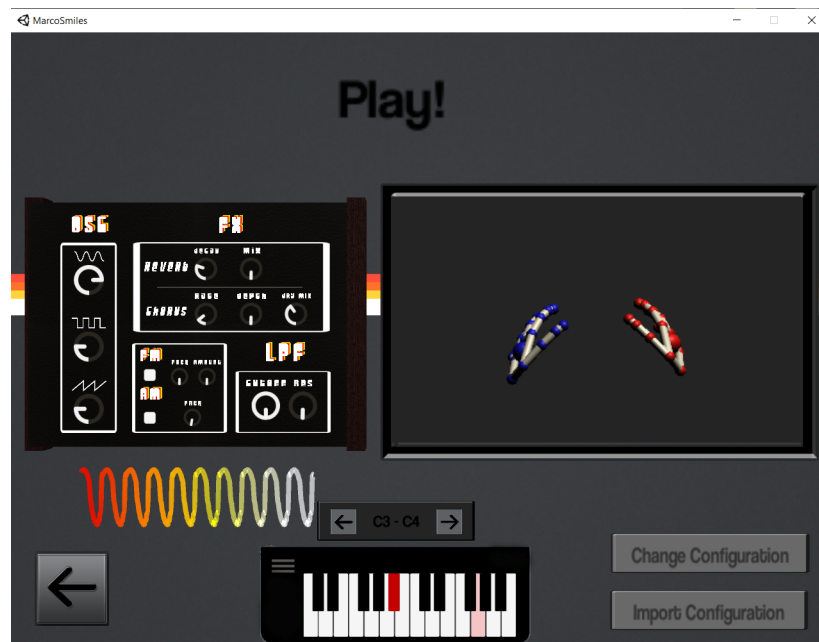


Figura 5.7: Play Scene

5. MODULO UNITY

Accanto alla finestra in cui sono visualizzate le mani è presente il pannello per controllare i parametri del Synth Virtuale. Il Synth ha la caratteristica di essere totalmente personalizzabile dall'utente, e dunque di cambiare timbro del suono, aggiungere modulazioni, filtri ed effetti. La personalizzazione è resa possibile grazie ad una interfaccia grafica intuitiva basata su Knobs, che vanno a simulare potenziometri di un sintetizzatore reale. Il design è ispirato al famoso sintetizzatore *MiniMoog*, nato del 1970 [30].



Figura 5.8: Sintetizzatore - MarcoSmiles

Sotto il pannello è presente un oscilloscopio che mostra la forma d'onda generata real-time. In basso a sinistra è posizionato un pulsante per tornare al menu principale, mentre in basso a sinistra sono presenti pulsanti che permettono rispettivamente di importare o esportare una configurazione del sistema.

CAPITOLO 6

CONCLUSIONI

6.1 RISULTATI E DISCUSSIONI

Il sistema MarcoSmiles 2.0 si pone come obiettivo l'accessibilità a ogni categoria di utente, mantenendo dei costi ristretti. Per il raggiungimento di questi due obiettivi abbiamo scelto il LeapMotion Controller come sensore da utilizzare per la rilevazione dei movimenti delle mani. Il costo del sensore è nettamente inferiore rispetto agli altri dispositivi per il tracking presenti sul mercato. Inoltre il suo utilizzo non risulta invasivo e ciò lo rende fruibile anche da persone con disabilità motorie. Considerando queste ragioni, il LeapMotion rappresenta il giusto compromesso tra prestazioni e costi nell'ambito dei dispositivi per il tracking. Rispetto alla versione precedente del sistema, l'interfaccia grafica (GUI) è stata sviluppata in Unity3D piuttosto che in Java. Le motivazioni che hanno portato a questa scelta sono numerose: l'ambiente Unity, da anni punto di riferimento nello sviluppo di videogiochi e sistemi di realtà aumentata, permette di sviluppare interfacce grafiche in maniera semplice ed intuitiva, offrendo alte prestazioni e garantendo un'elevata reattività agli input dell'utente. Inoltre grazie alla sua semplicità d'uso, un'interfaccia creata usando Unity3D risulta facilmente

manutenibile. Fondamentale è l'integrazione con il LeapMotion; quest'ultimo dispone di un set di API (LeapMotionSDK) in C# che permettono l'utilizzo del sensore all'interno di Unity, oltre che a numerosi moduli e assets realizzati per l'utilizzo esclusivo all'interno della piattaforma. La versione 2.0 del sistema MarcoSmiles rappresenta una vera evoluzione della precedente:

- Il design dell'interfaccia è stato migliorato, sono state aggiunte funzionalità per cambiare, importare, ed esportare una configurazione di sistema. Il sistema è stato riprogettato utilizzando tecnologie diverse, al fine di incrementare performance e reattività.
- Il processo di training risulta semplice e intuitivo, grazie all'aggiunta del Piano che permette di selezionare le note da allenare, e visualizzare quelle già allenate. Sono state anche inserite le funzioni per poter eliminare l'ultima configurazione delle mani registrata, e per poter eliminare il dataset nella sua interezza. Per quanto riguarda invece i risultati ottenuti dal training, per un dataset di 1 ottava, otteniamo un'accuratezza di circa il 90% nel riconoscere le posizioni, mentre per il dataset di 2 ottave scende al 75%.
- Il modulo di generazione del suono real-time è stato completamente riprogettato, ottenendo ottimi risultati in termini di reattività, rispetto alla versione precedente del sistema, in cui ciò rappresentava una delle maggiori problematiche. È stata anche aggiunta la possibilità di cambiare il timbro del suono generato, attraverso una serie di parametri che è possibile gestire all'interno dell'interfaccia grafica, in modo semplice ed efficace.

6.2 Lavori futuri

In questa sezione verranno esaminate alcune possibilità per quanto riguarda lo sviluppo di ulteriori funzioni da implementare all'interno del sistema:

6. CONCLUSIONI

- **Utilizzo di più sensori:** È possibile immaginare uno scenario in cui sono presenti più sensori, anche di tipologie diverse, in modo da catturare un maggior numero di informazioni sulle gestures dell'utente, al fine di migliorare sia le predizioni della rete neurale, aggiungendo nuove features di cui tener conto, che la modalità di interazione con il sistema stesso.
- **Utilizzo di altre features:** In virtù dell'osservazione precedente, si potrebbe pensare di aggiungere altre informazioni sulla configurazione delle mani. Ciò renderebbe il sistema altamente personalizzabile, aumentando di fatto il numero di gestures che l'utente potrebbe utilizzare.
- **Tool di Realtà Aumentata:** È possibile immaginare una possibile evoluzione del sistema MarcoSmiles come tool che sfrutta la realtà aumentata, in modo da permettere all'utente un'esperienza ancora più coinvolgente. La piattaforma Unity3D è la più rinomata ed utilizzata in questo ambito, dunque il lavoro per da svolgere risulterebbe facilitato.
- **Aumentare il volume del dataset:** Le analisi svolte nel modulo Machine learning hanno evidenziato come con l'aumentare del volume del dataset sia possibile ridurre il problema dell'*overfitting*, portando quindi un incremento della *accuracy* reale.
- **Sfruttare il Cloud per effettuare il training:** Considerando la potenza computazionale dei dispositivi di fascia media sul mercato, per minimizzare i tempi di training, si potrebbe implementare l'utilizzo di servizi Cloud. Sfruttando le grandi risorse computazionali messe a disposizione dalle loro infrastrutture, i tempi di addestramento verrebbero ridotti drasticamente.
- **Estensione del range di note suonabili:** Si potrebbe estendere il range di note oltre le 2 ottave. Questo porterebbe ad un'esperienza più vicina ad uno strumento reale.

6. CONCLUSIONI

- ***Modulare espressività del suono:*** Il senore potrebbe essere utilizzato per aumentare l'espressività del suono, modificando parametri del sintetizzatore attraverso specifici movimenti delle mani. Ad esempio, si potrebbe aumentare e diminuire il volume avvicinando ed allontanando le mani dal sensore Leap Motion, oppure utilizzare una mano per suonare note ed un'altra per modificare parametri.
- ***Estensione del range di note suonabili:*** Sfruttando il concetto di sintesi additiva, si potrebbero introdurre altre forme d'onda che riproducano un timbro che sia simile a strumenti reali, descrivendo lo spettro sonoro di questi ultimi.
- ***Preset:*** Si potrebbe aggiungere la possibilità di creare dei preset per il sintetizzatore, dando la possibilità all'utente di salvare i parametri di quest'ultimo.

BIBLIOGRAFIA

- [1] Maxine Sherrin, August de los Reyes: Predicting the past Web Directions, 2008
- [2] Ditte Hvas Mortensen, Natural User Interfaces – What are they and how do you design user interfaces that feel natural Interaction Design Foundation, 2020,
- [3] Computer Music - https://en.wikipedia.org/wiki/Computer_music
- [4] Pietro Grossi Computer Music - <http://www.hackerart.org/corsi/aba02/taddepera/computer-music.htm>
- [5] Ariana Grande Sings Harmonies with Mimu Gloves, San Jose April 12th, 2015 - <https://www.youtube.com/watch?v=sa6h6wZpE30>
- [6] Mi.Mu Gloves - <https://mimugloves.com/>
- [7] Natural User Interfaces to Support and Enhance Real-Time Music Performance - Rocco Zaccagnino, Roberto De Prisco, Delfina Malandrino, Gianluca Zaccagnino
- [8] Accessible Digital Musical Instruments — A Review of Musical Interfaces in Inclusive Music Practice, Emma Frid KTH Royal Institute of Technology, Lindstedtsvägen 3, 100 44 Stockholm, Sweden

BIBLIOGRAFIA

- [9] Theremin - <https://it.wikipedia.org/wiki/Theremin>
- [10] Mellotron - <https://it.wikipedia.org/wiki/Mellotron>
- [11] Omnichord - <https://it.wikipedia.org/wiki/Omnichord>
- [12] AUMI Software - <http://aumiapp.com/about.php>
- [13] KAIKu Music Gloves - <https://www.kaikumusicglove.com/>
- [14] Microsoft Kinect https://it.wikipedia.org/wiki/Microsoft_Kinect
- [15] Real-Time Musical Conducting Gesture Recognition Based on a Dynamic Time Warping Classifier Using a Single-Depth Camera - Fahn Chin-Shyurng, Shih-En Lee and Meng-Luen Wu, Department of Computer Science and Information Engineering, National Taiwan University of Science and Technology, Taipei 10607, Taiwan Dalian University of Technology
- [16] AeroMidi - <https://aeromidi.net/index.php>
- [17] Hand Gesture Recognition with Leap Motion - Youchen Du, Shenglan Liu, Lin Feng, Menghui Chen, Jie Wu
- [18] Virtual Piano - <https://gallery.leapmotion.com/virtual-piano-for-beginners/>
- [19] Natural User Interface - Next Mainstream Product User Interface, WeiyuanLiu, Guilin University Of Electronic Technology, School Of Art&Design Industiral Design, Guilin China
- [20] Leap Motion Controller by UltraLeap, https://www.ultraleap.com/datasheets/Leap_Motion_Controller_Datasheet.pdf
- [21] Unity Documentation <https://docs.unity3d.com/Manual/index.html>
- [22] Script lifecycle overview - <https://docs.unity3d.com/Manual/ExecutionOrder.html>
- [23] Wikipedia Feature Scaling https://en.wikipedia.org/wiki/Feature_scaling
- [24] wiimote figure https://en.wikipedia.org/wiki/Wii_Remote/

BIBLIOGRAFIA

- [25] Fondamenti di Acustica - <https://it.wikipedia.org/wiki/Suono>
- [26] Audio Digitale - https://it.wikipedia.org/wiki/Audio_digitale
- [27] Foto margine di errore Frequenze di campionamento
<https://www.headphonesty.com/2019/12/dacs-guide-part-1/>
- [28] tanhrelusigmoid figure <https://link.springer.com/article/10.1007/s11042-020-08740-w/figures/7>
- [29] <https://medium.com/@PABTennnis/how-to-create-a-neural-network-from-scratch-in-python-math-code-fd874168e955>
- [30] MiniMoog by Robert Moog - <https://it.wikipedia.org/wiki/Minimoog>
- [31] Python for Unity, <https://docs.unity3d.com/Packages/com.unity.scripting.python@2.0/manual/index.html>
- [32] IronPython - <https://ironpython.net/>
- [33] .NET Framework - <https://dotnet.microsoft.com/>